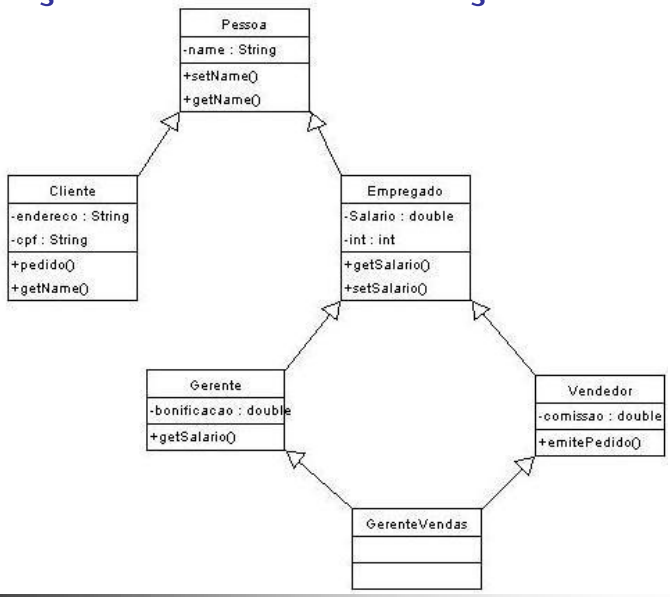
HERANÇA

- Herança é a capacidade de uma subclasse de ter acesso as propriedades da superclasse(também chamada classe base) relacionada a esta subclasse.
- Dessa forma os atributos e métodos de uma classe são propagados de cima para baixo em um diagrama de classes.
- Neste caso dizemos que a subclasse herda as propriedades e métodos da superclasse
- Os construtores da superclasse (classe base) não são herdados pela subclasse.
- A utilização da herança é um importante fator para a "reutilização de código"

HERANÇA

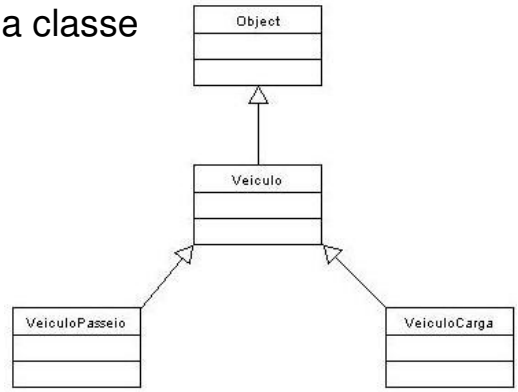
- A herança é uma capacidade característica das linguagens orientadas a objetos.
- A relação de herança entre duas classes é uma relação da seguinte forma: A "e um tipo de" B, onde A e B são classes. Caso esta relação entre as classes não puder ser construída, em geral, também não se tem uma relação de herança entre a classe A a partir da classe B.
- Exemplos: Um Carro de Passeio "é um tipo de " veículo; Um caminhão "é um tipo de" veículo; Um círculo "é um tipo de" figura; Um quadrado "é um tipo de" figura; Um vendedor "é um tipo de" Empregado; Um empregado "e um tipo de" pessoa.

HERANÇA - REPRESENTAÇÃO



HERANÇA - REPRESENTAÇÃO

- A Classe **Object** é a classe base da linguagem Java, ou seja, todas as outras classes herdam a classe **Object**
- Quanto mais alto na hierarquia de classe, mais generalizada é a classe



HERANÇA x USO

- Além da relação de herança entre as classes existe a relação de uso
  - HERANÇA

    classe A "é um tipo de" B
  - USO / AGREGAÇÃO (Relação de Conteúdo)

    classe D "contém" classe C"

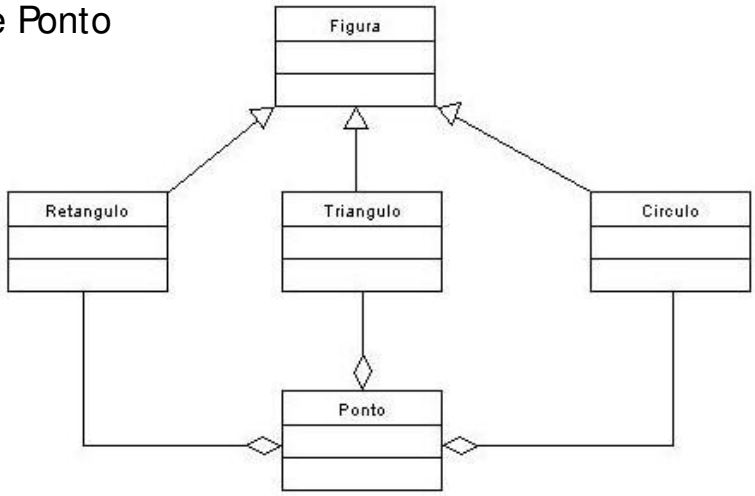
    classe D "usa" classe C"

    classe C "é parte da" classe D

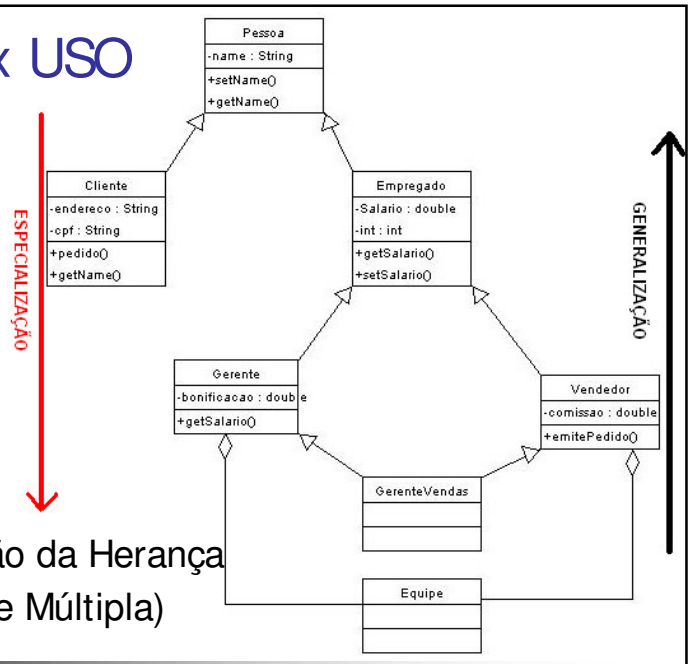
  Exemplo: Uma **Círculo** *contém* um **Ponto** central; Um **Triângulo** é *utiliza* três objetos da classe **Ponto**; Um **Ponto** *é parte* da classe **Quadrado**

HERANÇA x USO - REPRESENTAÇÃO

- Neste exemplo além da herança é representada a relação de uso, onde todos as outras figuras "usam" a classe Ponto



HERANÇA x USO



- Representação da Herança (Simples e Múltipla)

HERANÇA

- Para estabelecer uma relação de herança entre a classe a classe B e a classe A, deve ser utilizada keyword **extends**

  [public] [modTipo] class B ***extends*** A **[...]**
- Quanto mais alto na hierarquia de classe, mais generalizada é a classe
- Normalmente uma classe pode ser vista somente por outras classes no mesmo pacote. O modificador ***public*** indica que a classe será vista por qualquer outra classe
- Um **PACOTE** (**package**) consiste em um conjunto de classes relacionadas entre si.

HERANÇA

- Na criação de uma classe é possível utilizar modificadores de tipo que irão influenciar o comportamento da mesma.
- Modificador ***abstract:*** Indica que a classe não poderá ser instanciada
- Modificador ***final:*** Indica que a classe não poderá ser extendida

Exemplos:

public final class String ***extends*** Object

public class Triangulo ***extends*** Figura

class Empregado ***extends*** Pessoa

class VeiculoCarga ***extends*** Veiculo

abstract class Veiculo ***extends*** Object

HERANÇA – EXEMPLOS

```
class Pessoa {
  protected String name;
  //Construtor
  public Pessoa(){
    name = "";
  }
  public Pessoa(String n){
    name = n;
  }
  //Get Metodos
  String getName(){
    return name;
  }
```

```
//continuação...
//set Metodos
  void setName(String s){
    name = s;
  }
}
```

HERANÇA – EXEMPLOS

```
class Empregado extends
      Pessoa {
  protected double
      dSalario;
  //Construtor
  public Empregado(String
      n, double dS){
    name = n;
    dSalario = dS;
  }
```

```
//continuando...
//Get Metodos
  double getSalario(){
    return dSalario;
  }
//set Metodos
  void setSalario(double
      dS){
    dSalario = dS;
  }
}
```


HERANÇA – ACESSO A SUPERCLASSES

- A palavra reservada ***super***, permite acesso a métodos e construtores da superclasse
- **super(x1, x2, ..., xn)** - Permite a chamada do construtor da superclasse.
- Na utilização da herança é necessário a fim de chamar o construtor da superclasse, sempre que a superclasse não possui um construtor "default".
- Construtor "default" é aquele que não possui parâmetros
- Esta chamada deve ser a ***primeira*** dentro do construtor da classe. O construtor da classe imediatamente superior será chamado

HERANÇA – ACESSO A SUPERCLASSES

- **super.metodo(...)** - Permite que uma classe possa utilizar métodos definidos em sua superclasse
- Exemplos:

  String s;

  s = super.toString();

  Em algum método da classe empregado é possível a seguinte chamada:

  ...

  super.getName();

  ...

HERANÇA – EXEMPLOS

```
class Gerente extends
      Empregado {
  protected double
      dBonificacao;
  //Construtor
  public Gerente(String n,
      double dS, double
      dB){
    super(n,dS);
    dBonificacao = dB;
  }
double getBonificacao(){
    return dBonificacao;
  }
```

```
//continua...
  double getSalario(){
    return (dSalario +
      dBonificacao);
  }
  //set Metodos
  void
      setBonificacao(double
      dS){
    dBonificacao = dS;
  }
}
```

HERANÇA E POLIMORFISMO

- No exemplo anterior pode ser percebido que houve um polimorfismo no método **getSalario**.
- Definição do método na classe **Empregado**

```
double getSalario(){
  return dSalario;
}
```

- Definição no método na classe **Gerente**, que é subclasse de **Empregado**:

```
double getSalario(){
  return (dSalario + dBonificacao);
}
```

- O mesmo método apresenta um comportamento diferente para diferentes classes que possuem uma relação de herança

HERANÇA - CLASSES ABSTRATAS

- Algumas classes na hieraquia são tão gerais que nehum objeto será criado a partir delas. Neste caso a classe é dita **ABSTRATA**
- Uma classe abstrata não pode ser instanciada ou seja, não é possível criar objetos a partir da mesma
- A classe ABSTRATA é uma classe que está incompleta. Esta classe pode conter métodos abstratos que são aqueles métodos apenas declarados, mas que não foram implementados.
- Os métodos abstratos devem ser obrigatoriamente implementados nas subclasses.

HERANÇA - CLASSES ABSTRATAS

- O método abstrato contém apenas sua assinatura (nome, número e tipo dos seus parâmetros).
- Para a criação de classes e métodos abstratos deve ser utilizado o modificador de tipo "**abstract**"
- Classe CONCRETA é aquela a partir da qual objetos serão instanciados. Neste tipo de classe todos seus métodos devem ser, obrigatoriamente, definidos.

HERANÇA – CLASSES ABSTRATAS

```
abstract class Pessoa {
  protected String name;
  //Construtor
  //public Pessoa(){
  //  name = "";
  //}
  public Pessoa(String n){
    name = n;
  }
  //Get Metodos
  String getName(){
    return name;
  }
```

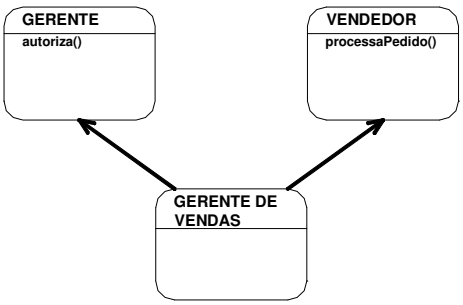
```
//continua...
//set Metodos
  void setName(String s){
    name = s;
  }
  abstract void
      printName();
}
```

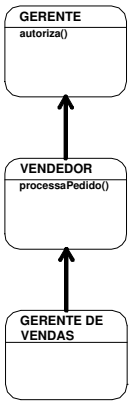
HERANÇA – CLASSES ABSTRATAS

```
class Empregado extends
      Pessoa {
  protected double
      dSalario;
  //Construtor
  public Empregado(String
      n, double dS){
    super(n);
    // name = n;
    dSalario = dS;
  }
  //Get Metodos
  double getSalario(){
    return dSalario; }
  //set Metodos
  void setSalario(double
      dS){
    dSalario = dS; }
  void printName(){
      System.out.println(
      "Empregado: " +
      name);
  }
}
```


HERANÇA MÚLTIPLA

- Existem casos em que uma classe pode herdar o comportamento de mais de uma classe. Neste caso temos a herança múltipla. Exemplo:

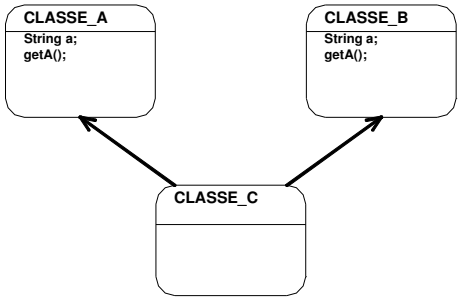




MODELAGEM INCORRETA

HERANÇA MÚLTIPLA

- Como implementar a herança múltipla:



- No exemplo acima, qual cópia do atributo **a** a classe CLASSE_C vai herdar? Qual método **getA()** vai utilizar?
- Java resolve este problema utilizando o conceito de "INTERFACES"

HERANÇA MÚLTIPLA

- Um método possui duas partes: sua assinatura e sua implementação
- Java não suporta a herança múltipla explicitamente, mas possui meios para que os efeitos da herança múltipla seja realizada de forma indireta utilizando o conceito de INTERFACES
- Através deste conceito uma classe pode herdar as assinaturas dos métodos, mas não a sua implementação.
- A implementação, deve por sua vez, ser definida na subclasse.

HERANÇA MÚLTIPLA

- Uma INTERFACE é definida através da palavra "**interface**" conforme mostrado a seguir:

  [public] interface B ***extends*** A

  Neste caso A deve ser outra interface.
- Exemplo – Definição da INTERFACE GerenteInt

```
interface GerenteInt{
  boolean autorizar();
}
```

- A indicação da herança múltipla é feita da seguinte forma:

[public] [modTipo] class B **[*extends*** A] **implements** C

HERANÇA MÚLTIPLA

- Na interface todos os métodos são abstratos e não possuem implentação apenas sua assinatura.
- A uma classe pode utilizar mais de uma interface em sua definição

HERANÇA MÚLTIPLA - EXEMPLO

```
class GerenteVendas
   extends Vendedor
   implements
   GerenteInt{
  protected String
   sRegiao;
  //Construtor
  public
   GerenteVendas(String
   n, double dS, double
   dC, String sReg){
    super(n, dS, dC);
    sRegiao = sReg;
  }
  //continua..
  void printName(){

   System.out.println("Ve
   ndedor: " + name);  }

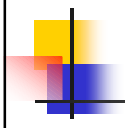
  public boolean
   autorizar(){
   System.out.println("Ve
   nda Autorizada");
    return true;  }
}
```


HERANÇA MÚLTIPLA

- Através da herança múltipla novos métodos, de diferentes classes, podem ser agregados a uma subclasse
- A herança através de interface não possibilita a reutilização do código, visto que o método herdado deve ser implementado para cada subclasse.
- ATRIBUTOS EM UMA INTERFACE: Em uma interface os atributos são implicitamente declarados como **static** e **final**.
- MÉTODOS EM UMA INTERFACE: Todos os métodos são abstratos, não sendo necessário a palavra "**abstract**"

HERANÇA MÚLTIPLA

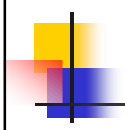
- **static:** Indica que existe apenas uma cópia do método ou varíavel, referenciados pela classe (método ou variável de classe)
- **final:** impede a modificação de um método, variável ou a especialização de uma classe.



HERANÇA MÚLTIPLA

- Exemplo: Métodos e Variáveis em interface

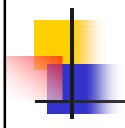
```
interface Cores{
  int RED = 1;
  int GREEN = 2;
  int BLUE = 3;
  void setCor(int c);
  int getCor();
}
```



HERANÇA MÚLTIPLA

- Uma classe abstrata que implenta alguma interface, deve conter a definição deste método. Exemplo:

```
abstract class Colorido implements Cores {
   int i;
   Colorido() {}
   public void setCor(int c){
      i = c;
   }
   …
```



HERANÇA MÚLTIPLA

- Diferenças entre classe Abstrata e Interface

CLASSE ABSTRATA	INTERFACE
Pode conter alguns métodos declarados como **abstract**	Somente pode ter métodos abstratos
Pode conter atributos **protected** e métodos **static**	Somente pode conter métodos **public**
Pode conter atributos do tipo **final** e "não-final"	Somente pode conter constantes (implicitamente são public final static)

